
VoltGuard

IoT Energy Monitoring Platform

Grupo 4

Name	Student Number	Main Contribution
Diogo Silva	108212	<i>oam-service / API Gateway / Orchestration</i>
Francisco Murcela	108815	<i>notifications-service</i>
Gonçalo Lima	108254	<i>anomalies-service / Observability</i>
Martim Carvalho	108749	<i>composer backend/frontend / Authentication</i>

<https://github.com/franciscomurcela/VoltGuard>



June 4, 2026

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Solution: VoltGuard	1
2	System Architecture	2
2.1	Overview	2
2.2	Technology Choices	3
2.3	Communication Patterns	3
3	Composer Service	4
3.1	Overview	4
3.2	Frontend	4
3.2.1	Technology & Build	4
3.2.2	Key Design Decisions	4
3.3	Backend	4
3.3.1	Technology & Role	4
3.3.2	Security	5
4	OAM Service	6
4.1	Overview	6
4.2	Technology	6
4.3	Data Model	6
4.4	API Surface	6
4.5	Kubernetes Storage	6
5	Anomaly Detection Service	7
5.1	Overview	7
5.2	Machine Learning Pipeline	7
5.3	Future Forecasting	7
5.4	Dynamic Calibration	7
5.5	API	7
5.6	Secrets & Configuration	8
6	Notifications Service	9
6.1	Overview	9
6.2	Technology Stack	9
6.3	Stored Data	9
6.4	Supported Channels and API Features	10
7	Kong API Gateway	11
7.1	Overview	11
7.2	Routing	11
7.3	High Availability	11

8	Deployment & Infrastructure	12
8.1	Kubernetes Overview	12
8.1.1	Persistent Volumes	12
8.1.2	Secrets	12
8.2	Load Balancing	12
8.2.1	Kong as the Single Entry Point	12
8.2.2	HorizontalPodAutoscaler	12
8.3	Observability	13
8.4	Stress Testing	13
8.5	Resilience Testing	14
9	Conclusions	16

1. Introduction

1.1 Problem Statement

Energy waste and undetected electrical faults are silent costs in industrial and urban environments. Traditional monitoring approaches are reactive: operators are notified *after* a failure has occurred, or at best during a scheduled inspection. This results in:

- Unplanned downtime that disrupts operations;
- Energy overconsumption that goes unnoticed until invoicing;
- Manual inspection overhead that scales poorly with the number of monitored sites;
- No consolidated view across geographically distributed installations.

As IoT sensor hardware becomes commodity, the bottleneck shifts from *data collection* to *data interpretation at scale*. The challenge is not deploying sensors — it is knowing what the sensors are saying, in real time, across potentially hundreds of distributed nodes.

1.2 Solution: VoltGuard

VoltGuard is an end-to-end IoT energy monitoring platform that turns raw sensor telemetry into actionable intelligence. The core value proposition is built on three pillars:

Monitor — Centralised, real-time ingestion of electrical measurements from all connected sensors in a single dashboard.

Detect — Automatic anomaly detection using time-series forecasting (Prophet), with no manual threshold configuration.

Respond — Immediate multi-channel alerting (email, SMS, WhatsApp) to the right person the moment an anomaly is confirmed.

2. System Architecture

2.1 Overview

VoltGuard follows a microservices architecture where each functional domain is isolated in its own deployable unit. All external traffic enters through a single ingress point: the Kong API Gateway, which is then forwarded to the appropriate backend service. Service-to-service calls use internal Kubernetes DNS and never traverse the public network.

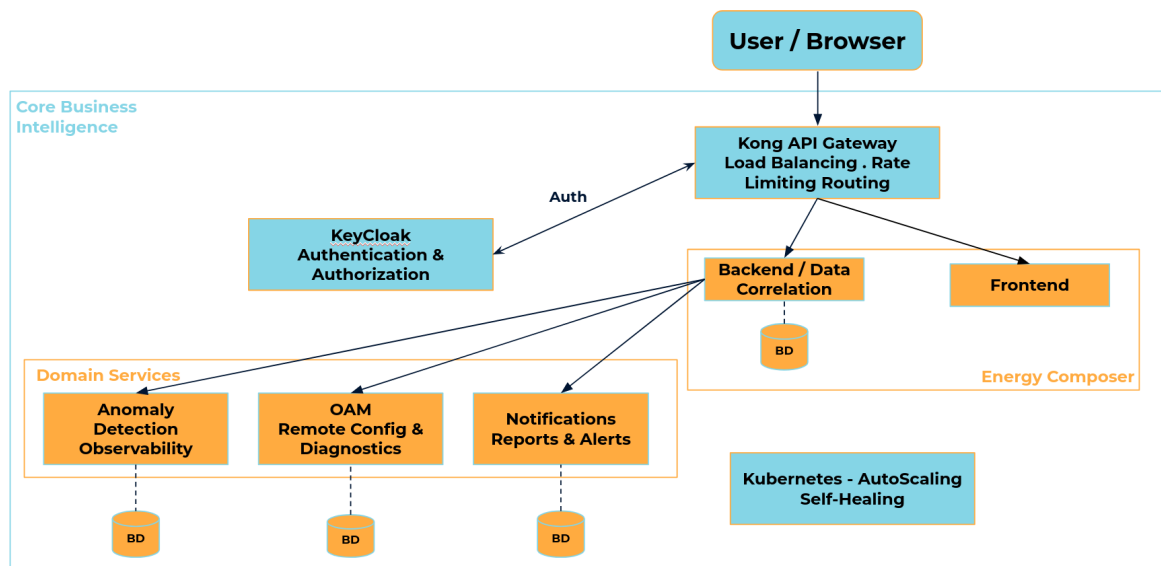


Figure 2.1: VoltGuard high-level architecture.

2.2 Technology Choices

Table 2.1: Technology stack per service

Component	Stack	Rationale
Composer Frontend	React + Vite	Component-driven SPA with fast HMR for development
Composer Backend	Node.js / Express	Lightweight SOA gateway; rich ecosystem for HTTP proxying
OAM Service	Rust / Axum / SQLx	Memory safety and throughput for sensor inventory operations
Anomaly Detection	Python / FastAPI	Native ecosystem for ML libraries (Prophet)
Notifications	Python / Connexion	OpenAPI-first design enforced by the Connexion framework
API Gateway	Kong 3.7 (DB-less)	Declarative config, zero-DB overhead, native K8s support
Authentication	Keycloak 25	Standards-compliant OIDC with PKCE; battle-tested in production
Observability	Prometheus + Grafana	De facto standard for cloud-native metrics collection and visualisation
Orchestration	Kubernetes	Self-healing pods, HPA, declarative infrastructure-as-code

2.3 Communication Patterns

- **Client** → **Kong**: HTTPS termination at the university Ingress Controller; Kong receives plain HTTP on port 8000.
- **Kong** → **Services**: Internal Kubernetes DNS.
- **Composer Backend** → **Peers**: Proxied REST calls with configurable timeout, retry count, and backoff.
- **Anomaly** → **Notifications**: Direct internal HTTP POST when an anomaly is confirmed.
- **Prometheus** → **Services**: Pull-based scrape every 15 seconds against each service's `/metrics` endpoint.

3. Composer Service

3.1 Overview

The Composer Service is the user-facing core of VoltGuard. It consists of two sub-components: a **React frontend** that provides the operator dashboard, and a **Node.js backend** that acts as a Service-Oriented Architecture (SOA) gateway — aggregating and normalising data from all peer microservices before presenting it to the client.

3.2 Frontend

3.2.1 Technology & Build

The frontend is built with **React 18** and **Vite**, served as a static Single-Page Application from an nginx container. Authentication is handled client-side using **Keycloak-js** with the PKCE flow, which requires a secure context (HTTPS or localhost). The Keycloak URL is injected at build time via a Vite environment variable:

```
1 docker build \  
2   --build-arg VITE_AUTH_DISABLED=false \  
3   --build-arg \  
4     VITE_KEYCLOAK_URL=https://grupo4-egs-deti.ua.pt/auth \  
5   -t registry.deti/grupo4-egs-deti.ua.pt/compositor-frontend:v5 \  
   composer-service/frontend/
```

Listing 3.1: Frontend Docker build arguments

3.2.2 Key Design Decisions

- Authentication variables are **build-time** (baked into the JS bundle), not runtime — this is a Vite constraint; `VITE_*` vars are resolved at `npm run build`.
- The nginx configuration sets `expires 1y`; `Cache-Control: public, immutable` for all static assets (JS, CSS, PNG), which maximises caching efficiency for content-hashed filenames.
- `VITE_AUTH_DISABLED` is independent of the backend's `AUTH_DISABLED` — both must be consistent or API calls will fail with 403.

3.3 Backend

3.3.1 Technology & Role

The backend is a **Node.js / Express** API that acts as the single integration point for all data consumers. It does not own any domain data directly; instead, it proxies and aggregates requests to OAM, Anomaly Detection, and Notifications.

3.3.2 Security

All endpoints are protected by Keycloak JWT validation. The backend verifies tokens against the Keycloak realm and rejects unauthenticated requests with HTTP 401. Sensitive configuration (client secrets, session keys, auth tokens) is loaded from Kubernetes secrets at runtime, never from plain environment variables in the deployment manifest.

4. OAM Service

4.1 Overview

The **Operations, Administration and Maintenance (OAM)** service manages the lifecycle of physical sensors deployed in the field. It is responsible for sensor registration, firmware inventory, and keepalive tracking.

4.2 Technology

The OAM service is written in **Rust** using the **Axum** web framework, with **SQLx** for async PostgreSQL access. Rust was chosen for its memory safety guarantees and predictable latency under load, important properties for a service that receives frequent keepalive heartbeats from a large sensor fleet.

4.3 Data Model

The service uses a **PostgreSQL 17** database managed by SQLx migrations. Persistent state includes:

- **Sensors** — unique identifier, location, firmware version, last-seen timestamp.
- **Firmware** — binary blobs stored on a persistent volume, with version metadata and checksum.
- **Keepalives** — timestamped heartbeat records used to detect sensor offline events.

4.4 API Surface

The OAM service exposes a REST API documented via OpenAPI (Swagger UI at `/swagger-ui`). Key endpoint groups:

Table 4.1: OAM service endpoint groups

Group	Path prefix	Description
Sensors	<code>/sensors</code>	CRUD for sensor registry
Firmware	<code>/firmware</code>	Upload, list, and assign firmware versions
Keepalive	<code>/keepalive</code>	Sensor heartbeat endpoint

4.5 Kubernetes Storage

Two `PersistentVolumeClaims` back the OAM service:

- `oam-postgres-data` — PostgreSQL data directory.
- `oam-firmware-storage` — Firmware binary blobs.

5. Anomaly Detection Service

5.1 Overview

The Anomaly Detection service is the intelligence layer of VoltGuard. It ingests sensor readings, models the expected baseline for each sensor, and flags readings that deviate significantly from that baseline.

5.2 Machine Learning Pipeline

The pipeline uses a robust time-series forecasting approach:

- **Prophet** (Meta / Facebook) - a decomposable time-series forecasting model that captures trend and seasonality. The pipeline splits incoming datasets using an **80/20 rule**: 80% of the historical data is used to train the model and establish the baseline, while the remaining 20% is then used for validation and **historical anomaly detection**, where any readings falling outside the generated confidence interval are flagged.

A reading is escalated to the Notifications service only if it falls outside the established Prophet confidence interval, ensuring proactive anomaly detection.

5.3 Future Forecasting

Beyond historical analysis, the persisted Prophet model unlocks proactive capabilities. The System can generate on-demand **forecasts** to project future energy consumption. This allows facility managers to anticipate energy spikes and plan accordingly, shifting the platform from a purely reactive monitor to a proactive intelligence tool.

5.4 Dynamic Calibration

Voltguard exposes configurable parameters to system operators. Through the frontend dashboard, users can dynamically adjust the **Prophet uncertainty interval**.

5.5 API

The service is built with **FastAPI** and exposes:

- **GET /measurements** — retrieves historical sensor readings.
- **GET /anomalies** — query confirmed anomaly events.
- **GET /forecasts/{sensor_id}** — generate future consumption forecasts on-demand using the persisted model.
- **PUT /models/{model_id}/config** — update the model sensitivity and configuration parameters.
- **POST /webhooks** — register an endpoint for asynchronous processing callbacks.
- **GET /metrics** — Prometheus metrics endpoint.

- `GET /docs` — auto-generated FastAPI interactive documentation.

5.6 Secrets & Configuration

The service uses a `vault_loader.py` module to pull sensitive configuration (such as the `ANOMALY_APP_TOKEN`) from HashiCorp Vault at startup. In the Kubernetes deployment, Vault is replaced by Kubernetes Secrets injected as environment variables.

6. Notifications Service

6.1 Overview

The Notifications service is a multi-channel alerting gateway that acts as an abstraction layer for operational communications. When the system's orchestrator (Composer) detects an anomaly event, it actively invokes this service by dispatching an HTTP POST request containing the payload. Upon receiving the request, the Notifications service evaluates user-defined routing profiles and executes the delivery process accordingly.

6.2 Technology Stack

The Notifications service is built upon a modern, asynchronous pythonic ecosystem designed for high throughput and strict contract enforcement. The complete technology stack comprises the following components:

- **Framework & API Enforcement:** Follows an **OpenAPI-first** approach using **Connexion**, a Python framework that dynamically generates the WSGI application layer.
- **Email Dispatch:** Implements native **SMTP** protocols through Python's built-in **smtplib** and **email** packages, supporting secure **TLS** encryption and integration with modern relay providers (e.g., Gmail App Passwords).
- **SMS Gateway:** Leverages the external **Twilio REST API** via the official **twilio** client library for transactional and operational mobile text messaging.
- **Extensible Architecture:** The core dispatch motor is decoupled from hardcoded credentials, dynamically consuming provider parameters from the database to allow seamless runtime registration or over-riding of additional communication channels without service downtime.

6.3 Stored Data

The service utilizes a **MongoDB** database to guarantee loose coupling and dedicated storage. Persistent state includes:

- **User Preferences** — Stores unique user identifiers, cryptographic secrets, channel targets (phone numbers, emails), and alert dispatch policies (immediate vs. digest).
- **Notifications & Digest Queue** — Holds operational logs of submitted payloads and a dedicated buffer queue for deferred alert aggregation.
- **Notification Audit** — Keeps a structured lifecycle event trail (e.g., **CREATED**, **DELIVERY_ATTEMPT**, **ABORTED_BY_PREFERENCE**) for diagnostic and compliance purposes .

6.4 Supported Channels and API Features

All channel integrations are decoupled from core business logic, supporting native email transmission and SMS integrations. Delivery is protected by an idempotency layer.

Table 6.1: Supported notification channels

Channel	Provider
Email	Native SMTP (<code>smtplib</code> with TLS)
SMS	Twilio SMS API

- **Idempotency Key:** The `POST /v1/notifications` endpoint enforces an idempotency mechanism using the `Idempotency-Key` HTTP header to prevent duplicate notifications during network retries.
- **Dynamic Preferences Link:** During the first notification attempt for a specific target, the system dynamically injects an opt-out URL appended with the user's unique cryptographic secret, enabling self-service profile management.

7. Kong API Gateway

7.1 Overview

Kong 3.7 operates in **DB-less** mode with a declarative configuration (`kong/kong.yml`). All routing rules, upstream targets, and plugins are defined in that single YAML file, making the gateway configuration fully reproducible and version-controlled.

7.2 Routing

Traffic arrives at the Kong proxy port and is routed to the appropriate upstream service based on the request path. Key routes:

Table 7.1: Kong routing table (Kubernetes deployment)

Path / Host pattern	Upstream service
/	Composer Frontend
/api/*	Composer Backend
/auth/*	Keycloak
/oam/*	OAM Service
/notify/*	Notifications Service
/anomaly/*	Anomaly Detection

7.3 High Availability

Kong is one of the two components managed by a **HorizontalPodAutoscaler**. The HPA maintains at least 2 Kong replicas at all times, scaling up to 5 under CPU pressure. This ensures that no single Kong pod failure interrupts service, and that the gateway capacity grows proportionally with incoming traffic.

8. Deployment & Infrastructure

8.1 Kubernetes Overview

All production workloads run in the namespace `tenant-grupo4-egs-deti-ua-pt` on the DETI cluster. The deployment is fully declarative: applying `k8s/deployment.yaml` and `k8s/observability.yaml` brings up the entire platform from scratch.

8.1.1 Persistent Volumes

Table 8.1: PersistentVolumeClaims

Name	Consumer
<code>composer-data</code>	Composer Backend (session data)
<code>oam-postgres-data</code>	OAM PostgreSQL
<code>oam-firmware-storage</code>	OAM firmware binaries
<code>notifications-db-data</code>	Notifications Service
<code>keycloak-postgres-data</code>	Keycloak PostgreSQL

8.1.2 Secrets

All sensitive values (database passwords, API tokens, client secrets, session keys) are stored in a Kubernetes Secret resource applied from `k8s/secrets.yaml` (gitignored). Services consume them as environment variables injected by the kubelet: no secrets ever appear in a container image or deployment manifest.

8.2 Load Balancing

8.2.1 Kong as the Single Entry Point

The university Ingress Controller terminates TLS and forwards all traffic to the Kong Service. Kong then load-balances across its replica pods using the standard Kubernetes Service round-robin mechanism.

8.2.2 HorizontalPodAutoscaler

Two HPAs are configured, both CPU-triggered:

Table 8.2: HorizontalPodAutoscaler configuration

Component	Min replicas	Max replicas	Metric
Kong Gateway	2	5	CPU utilisation
Composer Frontend	2	5	CPU utilisation

The minimum of 2 replicas for both components ensures that a single pod failure never causes a service outage, so there is always at least one healthy pod handling requests while Kubernetes reschedules the failed one.

8.3 Observability

The observability stack is deployed via `k8s/observability.yaml` and consists of:

- **Prometheus** (`prom/prometheus:v2.53.1`) — scrapes all application services every 15 seconds.
- **Grafana** (`grafana/grafana:11.1.0`) — serves pre-provisioned dashboards at `/grafana`.

Table 8.3: Prometheus scrape targets

Job name	Metrics path
anomaly-api	anomaly-api:8085/metrics
composer-backend	compositor-backend:8080/metrics
notifications-service	notifications-service:8083/metrics
oam-service	oam-service:8080/metrics

8.4 Stress Testing

Load testing was performed from a local machine using Apache Bench (`ab`) against the public HTTP endpoint:

```
1 ab -n 100000 -c 200 http://grupo4-egs-deti.ua.pt/
```

Listing 8.1: Stress test command

This generates 100,000 requests with a concurrency of 200. Under this load:

- The HPA scales Kong and Frontend pods up from 2 toward the 5-replica ceiling.
- Prometheus captures the CPU utilisation spike.
- Grafana dashboards provide real-time visibility into request rates and pod behaviour.
- Kong handles the burst with negligible connection failures.

Scaling events can be monitored live:

```

1 kubectl get events -n tenant-grupo4-egs-deti-ua-pt \
2   --field-selector involvedObject.kind=HorizontalPodAutoscaler
   --watch

```

Listing 8.2: Monitor HPA scaling events

```

diogozeca@diogolap-asusg15:~$ ab -n 100000 -c 200 http://grupo4-egs-deti.ua.pt/
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking grupo4-egs-deti.ua.pt (be patient)
Completed 10000 requests
Completed 20000 requests
Completed 30000 requests
Completed 40000 requests
Completed 50000 requests
Completed 60000 requests
Completed 70000 requests
Completed 80000 requests
Completed 90000 requests
apr_socket_recv: Connection timed out (110)
Total of 99978 requests completed

```

Figure 8.1: ApacheBench executing 100,000 requests at concurrency 200 against the live cluster — 99,978 requests completed.

```

Every 2.0s: kubectl get pods -n tenant-grupo4-egs-deti-ua-pt -l app=grupo4-kong diogolap-asusg15: Tue Jun 2 15:57:22 2026
NAME                                READY   STATUS    RESTARTS   AGE
grupo4-kong-69f799668f-4z2zc        0/1     Running   0           2s
grupo4-kong-69f799668f-7g58g        0/1     Running   0           25s
grupo4-kong-69f799668f-n10pc        0/1     ContainerCreating   0           1s
grupo4-kong-69f799668f-qh256        0/1     ContainerCreating   0           2s
grupo4-kong-69f799668f-vv10h        0/1     Running   0           11s

Every 2.0s: kubectl get pods -n tenant-grupo4-egs-deti-ua-pt -l app=grupo4-compositor diogolap-asusg15: Tue Jun 2 15:57:28 2026
NAME                                READY   STATUS    RESTARTS   AGE
grupo4-compositor-frontend-6f088ff6d7-4xmd        1/1     Running   0           27s
grupo4-compositor-frontend-6f088ff6d7-9yrf6s      1/1     Running   0           25h
grupo4-compositor-frontend-6f088ff6d7-9p032       1/1     Running   0           21s
grupo4-compositor-frontend-6f088ff6d7-j4tk9       1/1     Running   0           25h
grupo4-compositor-frontend-6f088ff6d7-x82xv       0/1     Running   0           12s

```

Figure 8.2: Kong CPU at 393% and Frontend at 359% of target — new pods enter ContainerCreating state as the HPA responds to the spike.

```

diogozeca@diogolap-asusg15:~$ kubectl get events -n tenant-grupo4-egs-deti-ua-pt --field-selector involvedObject.kind=HorizontalPodAutoscaler --watch
LAST SEEN   TYPE      REASON              OBJECT                                          MESSAGE
13m         Normal    SuccessfulRescale    horizontalpodautoscaler/compositor-frontend-hpa  New size: 5; reason: cpu resource utilization (percentage of request) above target
11m         Normal    SuccessfulRescale    horizontalpodautoscaler/compositor-frontend-hpa  New size: 2; reason: All metrics below target
13m         Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 5; reason: cpu resource utilization (percentage of request) above target
10m         Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 4; reason: All metrics below target
9m50s       Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 3; reason: All metrics below target
8m35s       Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 2; reason: All metrics below target
0s          Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 3; reason: cpu resource utilization (percentage of request) above target
0s          Normal    SuccessfulRescale    horizontalpodautoscaler/compositor-frontend-hpa  New size: 5; reason: cpu resource utilization (percentage of request) above target
0s          Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 5; reason: cpu resource utilization (percentage of request) above target
0s          Normal    SuccessfulRescale    horizontalpodautoscaler/compositor-frontend-hpa  New size: 2; reason: All metrics below target
0s          Normal    SuccessfulRescale    horizontalpodautoscaler/kong-hpa              New size: 4; reason: All metrics below target

```

Figure 8.3: HPA event stream: both HPAs scale up to 5 replicas under load, then step back down to 2–4 as CPU drops below target.

8.5 Resilience Testing

To validate that the platform tolerates pod failures during live traffic, a resilience test was performed with simultaneous pod termination under load. The terminal captures below show the full lifecycle as it happened.

Upon pod deletion:

1. The deleted pod transitions to **Terminating**.
2. Kubernetes immediately schedules a replacement pod, which enters **ContainerCreating** state.
3. The surviving Frontend replica continues serving requests.
4. The replacement pod reaches **Running** within approximately 10–20 seconds.
5. The HPA re-evaluates and restores the desired replica count.

The **ab** run completes with zero or negligible failed requests — any failures are limited to connections in-flight at the exact moment of termination — confirming that the platform is resilient to single-pod failures at the gateway layer.

```

diagozeca@diogolap-asusg15:~$ kubectl get pods -n t... diogolap-asusg15: Tue Jun 2 17:16:42 2026
NAME                                READY   STATUS    RESTARTS   AGE
grupo4-compositor-frontend-6fd86ffd47-8ztm6   1/1     Running   0           5m22s
grupo4-compositor-frontend-6fd86ffd47-d896t   1/1     Running   0           4m13s

diagozeca@diogolap-asusg15:~$ kubectl delete pod grupo4-compositor-frontend-6fd86ffd47-8ztm6 -n tenant-grupo4-egs-det1-ua-pt
pod "grupo4-compositor-frontend-6fd86ffd47-8ztm6" deleted from tenant-grupo4-egs-det1-ua-pt namespace

```

(a) Initial state: 2 Frontend pods **Running**; `kubectl delete pod` command issued.

```

diagozeca@diogolap-asusg15:~$ kubectl get pods -n t... diogolap-asusg15: Tue Jun 2 17:16:49 2026
NAME                                READY   STATUS    RESTARTS   AGE
grupo4-compositor-frontend-6fd86ffd47-8ztm6   1/1     Terminating   0           5m29s
grupo4-compositor-frontend-6fd86ffd47-d896t   1/1     Running        0           4m20s

diagozeca@diogolap-asusg15:~$ kubectl delete pod grupo4-compositor-frontend-6fd86ffd47-8ztm6 -n tenant-grupo4-egs-det1-ua-pt
pod "grupo4-compositor-frontend-6fd86ffd47-8ztm6" deleted from tenant-grupo4-egs-det1-ua-pt namespace

```

(b) Target pod transitions immediately to **Terminating**; second pod remains **Running**.

```

diagozeca@diogolap-asusg15:~$ kubectl get pods -n t... diogolap-asusg15: Tue Jun 2 17:17:02 2026
NAME                                READY   STATUS    RESTARTS   AGE
grupo4-compositor-frontend-6fd86ffd47-d896t   1/1     Running        0           4m33s
grupo4-compositor-frontend-6fd86ffd47-knpfw   1/1     Running        0           13s

diagozeca@diogolap-asusg15:~$ kubectl delete pod grupo4-compositor-frontend-6fd86ffd47-8ztm6 -n tenant-grupo4-egs-det1-ua-pt
pod "grupo4-compositor-frontend-6fd86ffd47-8ztm6" deleted from tenant-grupo4-egs-det1-ua-pt namespace
diagozeca@diogolap-asusg15:~$

```

(c) Replacement pod reaches **Running** within ≈ 13 s — cluster restored to 2 healthy replicas.

Figure 8.4: Pod self-healing lifecycle: from deletion through **Terminating** to a fresh **Running** replacement.

9. Conclusions

VoltGuard successfully delivers a production-grade IoT energy monitoring platform within the constraints of a university project. The key achievements are:

- **End-to-end functionality** — from sensor data ingestion, through ML-based anomaly detection, to real-time multi-channel alerting.
- **Resilient deployment** — Kubernetes HPA with a minimum of 2 replicas for both the gateway and the frontend ensures the platform survives pod failures without downtime.
- **Observability** — Prometheus and Grafana provide full visibility into the platform’s runtime behaviour, enabling proactive detection of performance degradation.
- **Security** — Keycloak OIDC with PKCE enforces authenticated access; secrets are never stored in version control or container images.
- **Developer experience** — declarative Kubernetes manifests, a local Docker Compose stack for development, and per-service Swagger UIs make the platform accessible to all team members.